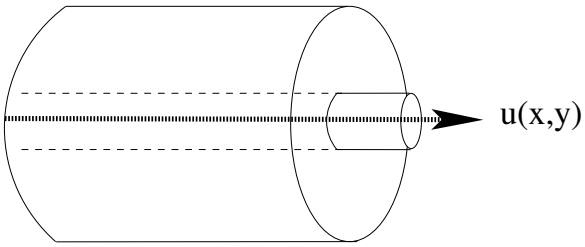
Prof. Dr. Christoph Pflaum
Benjamin Mann

Summer Term
2025

Simulation and Scientific Computing 2
Assignment 2

Propagation of Information within a Beam Waveguide



General Explanations

The information transport within a beam waveguide is described by the Helmholtz equation. The general three-dimensional problem can be reduced to two dimensions by using a time-harmonic approximation of the wave in propagation direction. This yields

$$-\Delta u - \left(k_0^2 n^2 - k_0^2 n_0^2\right) u = -2ik_0 n_0 \frac{\partial u}{\partial z} \quad \text{in } \Omega \tag{1}$$

with continuous functions u and n and two dimensional domain $\Omega = \{\mathbf{x} \in \mathbb{R}^2 \mid \|\mathbf{x}\|_2 < 1\}$. By defining $k := k_0\sqrt{n^2 - n_0^2}$ and $f := -2ik_0 n_0 \frac{\partial u}{\partial z}$, this can be simplified to

$$-\Delta u - k^2 u = f \quad \text{in } \Omega. \tag{2}$$

Assuming homogenous Neumann boundary conditions, the weak form of the problem reads

$$\text{find } u \in V \quad \text{such that} \quad a(u, v) = (f, v)_0 \quad \forall v \in V \tag{3}$$

with $V := H^1(\Omega)$. Here, $(\cdot, \cdot)_0$ denotes the L^2 scalar product, i.e.

$$(u, v)_0 = \int_\Omega u(\mathbf{x}) v(\mathbf{x}) \, \mathrm{d}\mathbf{x},$$

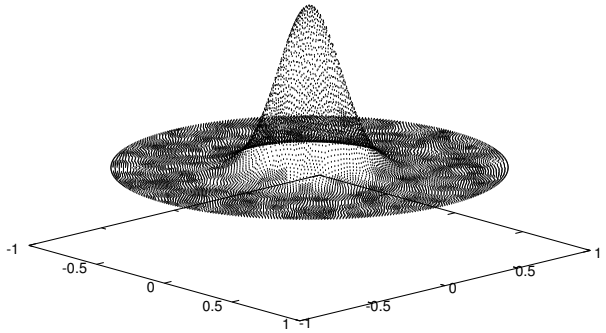
and the bilinear form $a \colon V \times V \to \mathbb{R}$ is defined by

$$a(u, v) := \int_\Omega \nabla u(\mathbf{x}) \cdot \nabla v(\mathbf{x}) - k(\mathbf{x})^2 u(\mathbf{x}) v(\mathbf{x}) \, \mathrm{d}\mathbf{x}. \tag{4}$$

The refractive index is significantly higher in the core of the waveguide than in the cladding. Let the coefficient k^2 be defined as

$$k^2(x, y) = (100 + \delta)e^{-50(x^2+y^2)} - 100, \tag{5}$$

for some $\delta \in (0, 100)$. Be aware that for small δ there might be round-off errors in the calculations.



Coefficient k^2 for $\delta = 0.01$.

In this assignment, we want to compute the beam propagation in the waveguide which is mainly described by the lowest order eigenmode of the associated eigenvalue problem

$$-\Delta u - k^2 u = \lambda u. \tag{6}$$

The right hand side in the weak form (3) is then given by $(\lambda u, v)_0$. Discretization by finite elements yields an eigenvalue problem in $\mathbb{R}^n$

$$A_h \mathbf{u} = \lambda M_h \mathbf{u}, \tag{7}$$

with stiffness and mass matrices A_h and M_h. Let $\varphi_1, ..., \varphi_n$ be the basis functions of the finite element space V_h. Then, these matrices are defined by

$$[A_h]_{ij} = a(\varphi_j, \varphi_i), \tag{8}$$

$$[M_h]_{ij} = (\varphi_j, \varphi_i)_0. \tag{9}$$

Inverse Power Iteration

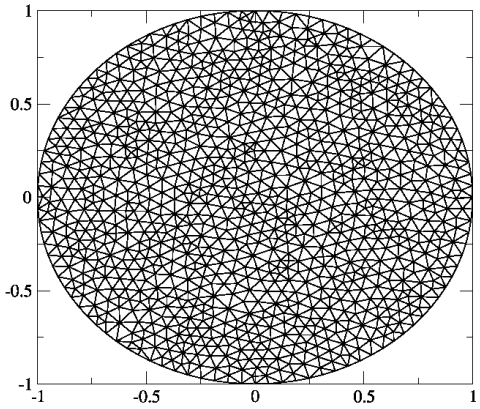
In order to find the smallest eigenvalue of (7), use the inverse power iteration:

1: **while** $|\frac{\lambda - \lambda_{old}}{\lambda}| > 10^{-10}$ **do**
2: $\quad \lambda_{old} = \lambda$
3: $\quad \mathbf{f} = M_h \mathbf{u}$
4: $\quad \mathbf{u} = A_h^{-1} \mathbf{f}$
5: $\quad \mathbf{u} = \frac{\mathbf{u}}{\|\mathbf{u}\|}$
6: $\quad \lambda = \frac{\mathbf{u}^T A_h \mathbf{u}}{\mathbf{u}^T M_h \mathbf{u}}$
7: **end while**

where $\|\cdot\|$ is the Euclidian norm in $\mathbb{R}^n$.

Mesh

For the domain discretizaion, we use an unstructured grid which is provided in the file `unit_circle.txt`. The file has the following format: First, vertex indices together with their coordinates are listed. Then, there follow by triples denoting the vertex indices of each triangular face element.



Assembly of Stiffness Matrix

For computing the local stiffness and mass matrices you may use the library **CoLSaMM** (**C**omputation **o**f **L**ocal **S**tiffness **a**nd **M**ass **M**atrices), available at

`https://www.cs10.tf.fau.de/research/software/expde/CoLSaMM/`

In order to apply **CoLSaMM**, just include the `CoLSaMM.h` file in your code. Then, proceed as follows:

1. In order to simplify the implementation, embed the **CoLSaMM** namespace into your program:

```
using namespace ::_CoLSaMM_;
```

2. Define the underlying reference element – in our case a triangle:

```
ELEMENTS::Triangle my_element;
```

3. For each triangle T_k, apply the following steps to compute the local matrix:

   (a) Pass the vertex coordinates to `my_element`:

```
// suppose coordsX[j], coordsY[j] contain the coordinate of
// the j-th vertex in the mesh and T[k] contains the vertex
// indices of the k-th triangle.
// required format: [x0, y0, x1, y1, x2, y2]
std::array<double, 6> vertices;
for (int i = 0; i < 3; ++i){
   vertices[2*i]   = coordsX[T[k][i]];
   vertices[2*i+1] = coordsY[T[k][i]];
}
my_element(vertices);
```

(b) Obtain the local matrix by calling, e.g.,

```
my_local_matrix =
   my_element.integrate(grad(v_()) * grad(w_()));
```

Note that in **CoLSaMM**, `v_` and `w_` denote the basis functions of the solution space and test space, respectively. The above example computes the local stiffness matrix corresponding to the bilinear form $a(u, v) = \int_\Omega \nabla u \cdot \nabla v$.

In order to introduce a user-defined external function of the kind

```
double my_func(double x, double y);
```

into the integrand, use

```
my_local_matrix =
   my_element.integrate(func<double>(my_func) * v_() * w_());
```

(c) Add the local matrix entries to the corresponding entries of your global stiffness / mass matrix.

Tasks

1. Read the data from the file `unit_circle.txt` and store the coordinates and triangles in suitable data structures.
2. Implement a function for computing $k(\mathbf{x})^2$, evaluate it at each vertex and write the resulting vector to a file `ksq.txt` in the following format:

   $x_1\ y_1\ k(x_1, y_1)^2$
   $x_2\ y_2\ k(x_2, y_2)^2$
   ...
   $x_n\ y_n\ k(x_n, y_n)^2$

   Display the result in `gnuplot` with the command `splot 'ksq.txt'` and compare it with the reference file on StudOn.
3. Implement a suitable data structure for the sparse matrices. Build the local stiffness and mass matrices with **CoLSaMM** and add their values to the correct positions in the matrices A_h and M_h.
4. Print the matrices A_h and M_h to the text files `A.txt` and `M.txt` and compare them with the reference files provided on StudOn. Use the following format:
   $i_1\ j_1\ A_{i_1,j_1}$
   $i_2\ j_2\ A_{i_2,j_2}$
   ...
   $i_n\ j_n\ A_{i_n,j_n}$

   Print the row and column indices in a lexicographical ordering, that is start by printing all matrix entries in the first row from left to right and continue in a row-wise fashion. Row and column indices start at 0. Also skip all zero entries in the matrix.

5. Provide a suitable solver for the matrix inversion in the inverse power iteration. You may use for example the conjugate gradient solver from the winter term, or the Red-Black Gauss-Seidel solver from the Multigrid assignment. To enable a flexible stopping criterion ϵ for your solver, the value for ϵ should be set on the command line.

6. Compute the beam propagation by applying the inverse power iteration as explained above. Write the resulting vector to a file `eigenmode.txt` in the following format:

   x_1 y_1 value$_1$
   x_2 y_2 value$_2$
   ...
   x_n y_n value$_n$

   Display the result in `gnuplot` with the command `splot 'eigenmode.txt'` and compare it with the reference file on StudOn. Note that due to numerical inaccuracies the results only match approximately. The corresponding eigenvalue though must match the reference eigenvalue rounded to *four decimal places* (when choosing a sufficiently small ϵ).

7. Make sure you use double precision floating-point calculations.

8. Please hand in your solution until **Tuesday, June 17, 2024** at 23:55. Make sure the following requirements are met:

   (a) The program should be compilable with a Makefile that you have to provide.

   (b) The program should compile without errors using the following g++ compiler flags:

   ```
   -Wall -std=c++20 -pedantic
   ```

   (c) The program should be callable by

   ```
   ./waveguide δ ϵ
   ```

   where δ is used in Eq. 5 and ϵ is the stopping criterion for the solver.

   (d) The program must output the approximation of the smallest eigenvalue λ after each iteration of the inverse power algorithm.

   (e) The solution should contain well commented source files, a fitting Makefile that satisfies all the conditions specified above and instructions how to use your program (e.g. in form of a short README file).

   (f) Submit your solution on StudOn as a team submission and include all your team members. Make sure that all required files are included in your submission!

Credits

In this assignment the credits are awarded in the following way:

1. Up to five points are awarded if your program correctly performs the above tasks and fulfills all of the above requirements. The correctness will be assessed by checking whether your program matches the reference outputs. Submissions with compile errors will lead to zero points! The computers in the computer science CIP will act as reference environments.

2. Bonus task: One bonus point is awarded if you implement the computation of local mass and stiffness matrices yourself, rather than using **CoLSaMM**.

   To compute the integrals on the reference element $\hat{T} = \mathrm{conv}\{(0,0),(1,0),(0,1)\}$, use the following quadrature rule:

$$\int_{\hat{T}} f(x,y)d(x,y) \approx \sum_{k=1}^{3} \omega_k f(x_k, y_k),$$

   where $\omega_1 = \omega_2 = \omega_3 = \frac{1}{6}$, and $(x_1, y_1) = \left(\frac{1}{6}, \frac{1}{6}\right)$, $(x_2, y_2) = \left(\frac{2}{3}, \frac{1}{6}\right)$, $(x_3, y_3) = \left(\frac{1}{6}, \frac{2}{3}\right)$.